

ARC-MEM Bridge v0.1

Architecture du système et tutoriel de fonctionnement

Architecture cible : ZORA_ARC V9.2 ARC-MEM-H

13 juin 2026

Résumé

ARC-MEM Bridge est une couche locale d'intégration entre Hermes Desktop, Hindsight, MemGraphRAG, H-MEM, PostgreSQL/PostGIS/pgvector et un moteur Python FastAPI. Ce document décrit l'architecture, la séparation des responsabilités, les flux de fonctionnement, les schémas PostgreSQL, les commandes d'exploitation et un tutoriel pratique pour diagnostiquer, démarrer, ingérer, récupérer et auditer la mémoire.

Table des matières

1	Objectif du système	3
1.1	Objectifs principaux	3
2	Vue d'ensemble	3
3	Etat cible retenu	4
4	Separation stricte des responsabilites	4
4.1	Hindsight	4
4.2	MemGraphRAG	4
4.3	H-MEM	5
5	Organisation PostgreSQL	5
5.1	Bases	5
5.2	Schemas ARC-MEM dans rag_arc	5
6	Application Python	5
6.1	Structure	5
6.2	Routes API	6
7	Tutoriel de fonctionnement	6
7.1	1. Activer l'environnement Python	6
7.2	2. Verifier PostgreSQL	6
7.3	3. Verifier Hindsight en PostgreSQL direct	7
7.4	4. Appliquer ou verifier les migrations	7
7.5	5. Demarrer l'API locale	7
7.6	6. Utiliser la CLI	7
7.7	7. Retenir un souvenir agentique	7
7.8	8. Rappeler un souvenir agentique	8
7.9	9. Ingerer un document probatoire	8
7.10	10. Recuperer un paquet de contexte	8
7.11	11. Auditer une reponse	8

8 Flux de retrieval	9
9 Securite et secrets	9
10 Commandes de validation	9
11 Etat valide au 13 juin 2026	9

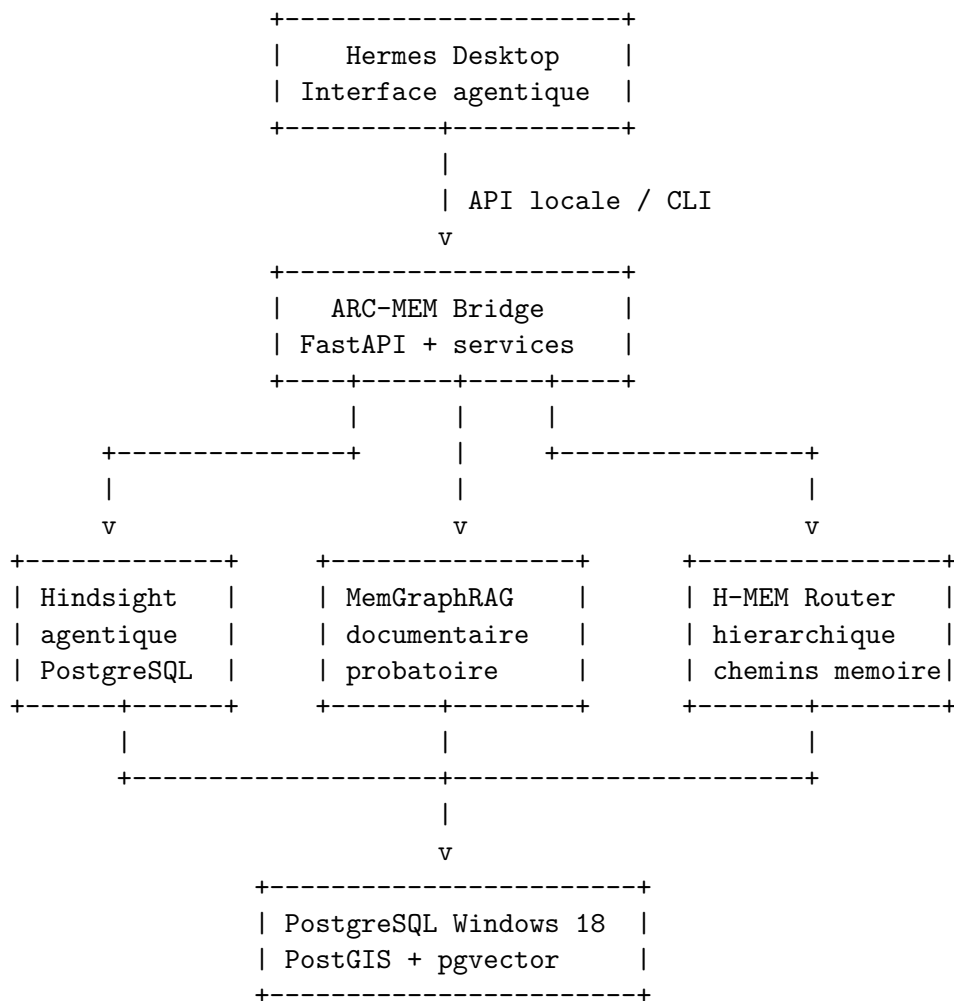
1 Objectif du système

ARC-MEM Bridge sert de pont logique local entre les composants de mémoire et d'orchestration. Il ne remplace ni Hermes, ni Hindsight, ni MemGraphRAG. Il fournit une API locale, une CLI et des services Python pour unifier l'accès aux mémoires, contrôler la provenance et appliquer la gouvernance ZORA ARC.

1.1 Objectifs principaux

- connecter proprement PostgreSQL depuis WSL ;
- garder Hindsight comme mémoire agentique persistante ;
- garder MemGraphRAG comme mémoire documentaire probatoire ;
- utiliser H-MEM comme routeur hiérarchique, sans base séparée ;
- exposer une API locale pour Hermes ;
- appliquer des contrôles ZORA ARC : TASKCARD, preuve, audit, gates ;
- éviter toute fuite de secrets dans les logs, tests et documentation.

2 Vue d'ensemble



3 Etat cible retenu

L'installation actuelle retient le mode `postgresql_direct` pour Hindsight. Ce choix abandonne explicitement PostgreSQL embedded, qui est fragile sur Windows avec des locales regionales non ASCII.

Composant	Etat cible
Hermes Desktop	Interface agent. Ne doit pas etre modifie brutalement. Les fichiers de configuration doivent etre sauvegardes avant toute modification.
Hindsight	Memoire agentique persistante en mode <code>postgresql_direct</code> . Stocke preferences, decisions, contexte de travail et resumes intersessions dans <code>hindsight_hermes</code> .
MemGraphRAG	Memoire documentaire probatoire dans les schemas ARC-MEM. Stocke documents, passages, entites, faits, relations, conflits et preuves.
H-MEM	Couche de routage hierarchique. Les niveaux sont <code>domain</code> , <code>theme</code> , <code>subtheme</code> , <code>entity</code> , <code>fact</code> , <code>passage</code> .
PostgreSQL	Serveur central Windows 18 accessible depuis WSL via 172.21.96.1:5432. Extensions actives : <code>vector</code> , <code>postgis</code> , <code>pg_trgm</code> , <code>pgcrypto</code> .
ZORA ARC	Gouvernance methodologique : TASKCARD, score routeur, registre de preuves, audit, niveaux de criticite.

4 Separation stricte des responsabilites

4.1 Hindsight

Hindsight reste reserve a la memoire agentique persistante :

- preferences utilisateur ;
- decisions de projet ;
- historique de travail ;
- resumes intersessions ;
- contexte Hermes ;
- faits agentiques courts et recherchables.

Il ne doit pas recevoir les PDF complets ni les gros chunks documentaires. Dans l'etat actuel, Hindsight est connecte directement a PostgreSQL dans la base `hindsight_hermes`. Le daemon Hindsight et PostgreSQL embedded ne sont pas necessaires pour cette voie.

4.2 MemGraphRAG

MemGraphRAG est reserve a la memoire documentaire probatoire :

- documents sources ;
- chunks ;
- passages ;
- entites ;
- faits documentaires ;
- conflits ;
- preuves et provenance.

La chaine de tracabilite visee est :

source -> chunk -> passage -> fait -> citation -> reponse

4.3 H-MEM

H-MEM ne cree pas une base de memoire parallele. Il route les requetes vers les bons niveaux d'abstraction et les bons objets. Les noeuds H-MEM peuvent pointer vers :

- `zora.documents`;
- `zora.chunks`;
- `memgraph.mem_entities`;
- `memgraph.mem_facts`;
- `memgraph.mem_passages`;
- des souvenirs Hindsight par reference externe.

5 Organisation PostgreSQL

5.1 Bases

Base	Role
<code>rag_arc</code>	Base ARC-MEM Bridge. Contient les schemas <code>zora</code> , <code>memgraph</code> , <code>hmem</code> .
<code>hindsight_hermes</code>	Base Hindsight directe. Contient la memoire agentique deja migree : faits, sessions, entites, documents, memories.

5.2 Schemas ARC-MEM dans `rag_arc`

Schema	Tables principales
<code>zora</code>	<code>taskcards</code> , <code>arc_runs</code> , <code>arc_audit_log</code> , <code>evidence_registry</code> , <code>documents</code> , <code>chunks</code> , <code>citations</code> .
<code>memgraph</code>	<code>mem_schemas</code> , <code>mem_entities</code> , <code>mem_facts</code> , <code>mem_passages</code> , <code>mem_bridges</code> , <code>mem_conflicts</code> , <code>v_graphe_ppr</code> .
<code>hmem</code>	<code>memory_layers</code> , <code>memory_nodes</code> , <code>memory_edges</code> , <code>positional_indexes</code> , <code>routing_paths</code> .

6 Application Python

6.1 Structure

```
arc-mem-bridge/
  app/
    main.py
    config.py
    db.py
  api/
    health.py
    memory.py
    ingest.py
    retrieve.py
    audit.py
  services/
```

```

hindsight_client.py
memgraph_service.py
hmem_router.py
zora_arc_service.py
retrieval_service.py
ppr_service.py
hermes_bridge.py
migrations/
  001_init_extensions.sql
  002_init_zora_schema.sql
  003_init_memgraph_schema.sql
  004_init_hmem_schema.sql
  005_views_and_indexes.sql
cli/
  arc_mem.py
scripts/
  check_postgres.py
  apply_migrations.py
  diagnose_hermes.py
  diagnose_hindsight.py
  smoke_test.py

```

6.2 Routes API

Route	Role
GET /health	Retourne le statut de l'application, PostgreSQL, extensions et Hindsight.
POST /memory/retain	Retient un souvenir agentique dans Hindsight. En mode direct, insere dans <code>hindsight_hermes.facts</code> .
POST /memory/recall	Recherche des souvenirs agentiques Hindsight.
POST /ingest/document	Enregistre un document dans <code>zora</code> , le decoupe en chunks et prepare l'extraction probatoire.
POST /retrieve	Construit un paquet de contexte structure : TASKCARD, contexte Hindsight, chemins H-MEM, faits, passages, citations.
POST /audit	Verifie qu'une reponse proposee contient sources, citations, limites, risques et coherence avec la TASKCARD.

7 Tutoriel de fonctionnement

7.1 1. Activer l'environnement Python

```

cd /mnt/c/Users/Stephane_Arnoux/arc-mem-bridge
. .venv/bin/activate

```

7.2 2. Verifier PostgreSQL

```
python scripts/check_postgres.py
```

Resultat attendu :

```

{"ok": true
 "database": "rag_arc"
 "vector": true
}

```

```
"postgis": true
"pg_trgm": true
"pgcrypto": true
```

7.3 3. Verifier Hindsight en PostgreSQL direct

```
python scripts/diagnose_hindsight.py
```

Resultat attendu :

```
"ok": true
"mode": "postgresql_direct"
"facts": 661
"sessions": 212
"entities": 14
```

7.4 4. Appliquer ou verifier les migrations

Les migrations sont idempotentes. Pour les auditer sans execution :

```
python scripts/apply_migrations.py --dry-run
```

Pour les appliquer :

```
python scripts/apply_migrations.py
```

7.5 5. Demarrer l'API locale

```
uvicorn app.main:app --host 127.0.0.1 --port 8791
```

Puis tester :

```
curl http://127.0.0.1:8791/health
```

7.6 6. Utiliser la CLI

```
python -m app.cli.arc_mem health
python -m app.cli.arc_mem check-postgres
python -m app.cli.arc_mem diagnose-hermes
python -m app.cli.arc_mem diagnose-hindsight
python -m app.cli.arc_mem smoke-test
```

7.7 7. Retenir un souvenir agentique

```
curl -X POST http://127.0.0.1:8791/memory/retain \
-H "Content-Type: application/json" \
-d '{
  "text": "ARC-MEM Bridge utilise Hindsight en postgresql_direct.",
  "metadata": {"source": "tutorial", "type": "configuration"}
}'
```

En mode `postgresql_direct`, cette route insere une ligne dans `hindsight_hermes.facts`. Elle ne duplique pas ce souvenir dans MemGraphRAG.

7.8 8. Rappeler un souvenir agentique

```
curl -X POST http://127.0.0.1:8791/memory/recall \
-H "Content-Type: application/json" \
-d '{"query": "postgresql_direct", "limit": 5}'
```

7.9 9. Ingerer un document probatoire

```
curl -X POST http://127.0.0.1:8791/ingest/document \
-H "Content-Type: application/json" \
-d '{
  "title": "Note architecture ARC-MEM",
  "content": "MemGraphRAG conserve les passages et les faits documentaires.",
  "source_uri": "local://tutorial",
  "metadata": {"domain": "architecture"}
}'
```

Cette operation ecrit dans `zora.documents` et `zora.chunks`. Les embeddings et l'extraction LLM restent des etapes configurees separement.

7.10 10. Recuperer un paquet de contexte

```
curl -X POST http://127.0.0.1:8791/retrieve \
-H "Content-Type: application/json" \
-d '{
  "query": "Comment ARC-MEM separe Hindsight et MemGraphRAG ?",
  "expected_evidence_level": "standard",
  "limit": 8
}'
```

La reponse n'est pas une reponse finale longue. Elle contient un `ContextBundle` :

- `memories` : contexte agentique Hindsight ;
- `route_paths` : chemins H-MEM ;
- `facts` : faits MemGraphRAG ;
- `passages` : chunks documentaires ;
- `citations` : provenance ;
- `warnings` : limites et indisponibilites.

7.11 11. Auditer une reponse

```
curl -X POST http://127.0.0.1:8791/audit \
-H "Content-Type: application/json" \
-d '{
  "taskcard": {"title": "Audit exemple"},
  "proposed_answer": "Reponse courte",
  "sources": [],
  "citations": [],
  "limits": [],
  "risks": []
}'
```

Si sources, citations, limites ou risques manquent, l'audit retourne des `findings`.

8 Flux de retrieval

1. L'utilisateur ou Hermes appelle `POST /retrieve`.
2. ZORA ARC cree une TASKCARD minimale.
3. Le routeur ZORA ARC calcule les signaux `Ct`, `Cs`, `Rk`, `Ex`, `Op`, `Tp`.
4. Hindsight fournit le contexte agentique via PostgreSQL direct.
5. H-MEM choisit un chemin de routage hierarchique.
6. MemGraphRAG recherche les faits/passages probatoires.
7. Le service de retrieval assemble un `ContextBundle`.

9 Securite et secrets

- Ne jamais exposer `.env`.
- Ne jamais afficher de cle complete dans les logs.
- Les motifs `nvapi-*`, `sk-*`, tokens, passwords et secrets sont masques par la couche de logging.
- Les configurations Hermes/Hindsight sont sauvegardees avant modification.
- PostgreSQL embedded est abandonne pour eviter les erreurs de locale Windows et les locks de fichiers.

10 Commandes de validation

```
python scripts/check_postgres.py
python scripts/diagnose_hindsight.py
python scripts/apply_migrations.py --dry-run
python -m pytest
python -m app.cli.arc_mem health
uvicorn app.main:app --host 127.0.0.1 --port 8791
```

11 Etat valide au 13 juin 2026

Controle	Resultat
PostgreSQL rag_arc	OK
Extensions	vector, postgis, pg_trgm, pgcrypto OK
Hindsight	OK en postgresql_direct
Memoire Hindsight	661 faits, 212 sessions, 14 entites
Tests Python	8 tests passent
API locale	/health repond OK